

Cenário

Vamos modelar um endereço em um sistema de pedidos. O endereço (rua, número, cidade) é um **Value Object**, porque:

- Não tem identidade única.
- A troca de um endereço por outro com os mesmos valores não muda o sistema.

```
public class Address
{
    public string Street { get; }
    public string City { get; }
    public string ZipCode { get; }

    public Address(string street, string city, string zipCode)
    {
        if (string.IsNullOrEmpty(street) ||
            string.IsNullOrEmpty(city) ||
            string.IsNullOrEmpty(zipCode))
        {
            throw new ArgumentException("All address fields must be
provided.");
        }

        Street = street;
        City = city;
        ZipCode = zipCode;
    }

    // Comparação por valor
    public override bool Equals(object obj)
    {
        if (obj is not Address other)
        {
            return false;
        }

        return Street == other.Street &&
            City == other.City &&
            ZipCode == other.ZipCode;
    }

    public override int GetHashCode()
    {
        return HashCode.Combine(Street, City, ZipCode);
    }
}
```

Uso do Value Object

```
public class Order
{
    public int Id { get; set; }
    public Address ShippingAddress { get; private set; }

    public Order(int id, Address shippingAddress)
    {
        Id = id;
        ShippingAddress = shippingAddress ?? throw new
ArgumentNullException(nameof(shippingAddress));
    }

    public void ChangeShippingAddress(Address newAddress)
    {
        if (newAddress == null)
        {
            throw new ArgumentNullException(nameof(newAddress));
        }

        ShippingAddress = newAddress;
    }
}
```

Exemplo de Código Cliente

```
var address1 = new Address("123 Main St", "Springfield", "12345");
var address2 = new Address("123 Main St", "Springfield", "12345");
var address3 = new Address("456 Elm St", "Shelbyville", "67890");

Console.WriteLine(address1.Equals(address2)); // True (mesmos valores)
Console.WriteLine(address1.Equals(address3)); // False (valores
diferentes)

// Usando no pedido
var order = new Order(1, address1);
Console.WriteLine($"Order Address: {order.ShippingAddress.Street}");

// Alterando o endereço
order.ChangeShippingAddress(address3);
Console.WriteLine($"Updated Order Address:
{order.ShippingAddress.Street}");
```

Vantagens do Value Object

1. **Imutabilidade:** Garante que os dados não sejam alterados acidentalmente.
2. **Consistência:** Comparações baseadas em valor reduzem erros.
3. **Modelagem rica:** Representa conceitos do domínio com mais precisão.

Quando usar Value Objects?

- Para representar pequenos conceitos do domínio como:
 - Endereço, CPF, Nome completo, etc.
- Quando a identidade única não é necessária.